

Guide de collaboration GitHub – Workflow des Pull Requests (PR)

Introduction

Dans ce projet, la branche `main` est protégée.

Cela signifie que :

- ❌ Il est interdit de développer directement sur `main`.
- ❌ Il est interdit de faire un `git push` directement sur `main`.
- ❌ Aucun développeur ne peut fusionner sa propre Pull Request.
- ✅ Toutes les modifications doivent passer par une Pull Request (PR).
- ✅ Le chef de projet est responsable de la relecture, de la validation et du Merge des Pull Requests.

L'objectif est de garantir la qualité du code, d'éviter les erreurs et de conserver un historique propre.

Le Workflow d'une Pull Request (PR)

Étape 1 : Mettre son projet à jour

Avant de commencer une nouvelle tâche, récupérez toujours la dernière version du projet.

```
git checkout main
git pull origin main
```

Étape 2 : Créer une branche dédiée

Une branche = une seule fonctionnalité ou une seule correction.

Ne jamais travailler directement sur `main`.

Exemple :

```
git checkout -b feature/user-login
```

Explication des conventions de branches (IMPORTANT)

Comme l'équipe est francophone, voici comment comprendre les noms de branches :

-  **feature/** → nouvelle fonctionnalité
 Utilisé quand tu ajoutes quelque chose de nouveau.

Exemples :

- `feature/login-utilisateur`
- `feature/tableau-de-bord`

- `feature/paiement-stripe`
- 🟡 **fix/** → correction de bug
👉 Utilisé quand tu corriges un problème.
Exemples :
 - `fix/erreur-connexion`
 - `fix/mot-de-passe-invalid`
 - `fix/bouton-qui-ne-marche-pas`
- 🟣 **refactor/** → amélioration du code sans changer le fonctionnement
👉 Utilisé pour nettoyer ou améliorer le code.
Exemples :
 - `refactor/service-auth`
 - `refactor/api-utilisateurs`
- 🟢 **docs/** → documentation
👉 Utilisé pour modifier la documentation.
Exemples :
 - `docs/readme`
 - `docs/guide-installation`
- 🟡 **chore/** → tâches techniques
👉 Utilisé pour les tâches internes (non fonctionnelles).
Exemples :
 - `chore/update-dependencies`
 - `chore/config-eslint`

Étape 3 : Développer et faire des commits

Pendant le développement, fais des commits clairs.

```
git add .  
git commit -m "feat: ajout du formulaire de connexion"
```

📌 Explication des messages de commit (très important)

Les commits suivent aussi des conventions simples :

- ✅ **feat:** → nouvelle fonctionnalité
`feat: ajout du login utilisateur`

-  **fix:** → correction de bug
`fix: correction erreur de connexion`
-  **refactor:** → amélioration du code
`refactor: simplification du service auth`
-  **docs:** → documentation
`docs: mise à jour du README`
-  **chore:** → maintenance technique
`chore: mise à jour des dépendances`

❌ Mauvais exemples (à éviter)

- `update`
- `fix bug`
- `modif`
- `test`
- `code`

👉 Ces messages ne disent pas ce qui a été fait.

Étape 4 : Envoyer la branche

```
git push -u origin feature/user-login
```

Étape 5 : Créer une Pull Request

- Aller sur GitHub
- Cliquer sur **Compare & Pull Request**
- Vérifier que la cible est `main`
- Ajouter un titre clair
- Créer la PR

Étape 6 : Validation par le chef de projet

- ❌ Tu ne peux pas merger toi-même
-  Le chef de projet valide ou refuse
- 💬 Il peut demander des modifications

Bien rédiger sa Pull Request

Titre

Utilise les mêmes conventions que les commits :

- `feat: ajout login utilisateur`
- `fix: correction bug connexion`

Description

- **Quoi ?** Ce que tu as fait.
- **Pourquoi ?** Pourquoi tu l'as fait.
- **Comment tester ?** Comment vérifier ton travail.

Code Review

Le chef de projet peut commenter ton code.

Exemples :

- "Cette fonction peut être simplifiée"
- "Attention à la performance ici"
- "Renomme cette variable"

👉 Ce n'est pas une critique personnelle, c'est pour améliorer le projet.

Corriger une review

Tu modifies ton code dans la même branche :

```
git add .
git commit -m "fix: corrections review"
git push
```

Résoudre un conflit Git

Si GitHub affiche un conflit :

```
git checkout main
git pull origin main
git checkout feature/user-login
git merge main
```

Puis corriger dans ton éditeur.

Ensuite :

```
git add .
git commit -m "chore: résolution conflit"
git push
```

Après le Merge

```
git checkout main
git pull origin main
```

```
git branch -d feature/user-login  
git push origin --delete feature/user-login
```

Bonnes pratiques

- Toujours travailler sur une branche
- Faire des commits petits et clairs
- Utiliser les conventions feat / fix / refactor / docs / chore
- Ne jamais push sur `main`
- Toujours passer par une PR

Résumé du workflow

1. pull main
2. créer une branche feature/*
3. coder
4. commit
5. push
6. PR
7. review chef de projet
8. merge
9. suppression branche

Conclusion

Dans ce projet, tout passe par des branches et des Pull Requests.
Les conventions (branches + commits) permettent :

- une meilleure organisation
- un code plus propre
- une collaboration claire
- un historique compréhensible pour toute l'équipe francophone